



Departamento de Informática e Matemática Aplicada – DIMAp

Observabilidade em Sistemas Distribuídos

Prof. Nélio Cacho

Universidade Federal do Rio Grande do Norte
neliocacho@dimap.ufrn.br

Hora de silenciar o celular...



- Manter o telefone celular sempre desligado/silencioso quando estiver em sala de aula;
- Nunca atender o celular na sala de aula.

O que é Observabilidade?

- *Observabilidade é a capacidade de entender o estado interno de um sistema a partir de seus outputs externos (logs, métricas e rastreamentos).*
- Atua na coleta e análise dos seguintes conjuntos de dados:
 - **Métricas:** dados quantitativos (ex.: uso de CPU, throughput).
 - **Traces (rastreamentos):** fluxo de chamadas entre componentes distribuídos.
 - **Logs:** registros de eventos(Não Iremos detalhar neste curso).

Observabilidade em Arquitetura Monolítica

- Um único processo (JVM) agrupa tudo.
- Monitoramento concentrado em:
 - Logs unificados.
 - Métricas globais do processo.
 - Stack traces mais fáceis de seguir.
 - Diagnóstico mais simples (sem hops de rede).

Observabilidade em Microserviços

- **Características:**

- Muitos processos independentes.
- Chamadas distribuídas (REST, gRPC, mensageria).
- Cada serviço expõe suas próprias métricas e logs.
- Necessidade de correlação de eventos entre serviços.

- **Desafios adicionais:**

- Traçar requisições ponta a ponta.
- Unificar logs dispersos.
- Latências de rede.

Observabilidade em Diferentes Ambientes

Aspecto	Monólito	Microserviços
Visibilidade	Centralizada	Distribuída por serviços
Tracing	Local	Distribuído e mais complexo
Coleta de logs	Em um único arquivo/servidor	Múltiplos pontos de coleta
Complexidade	Baixa	Alta (necessita sistemas de agregação/correlação)
Ferramentas	Actuator, Micrometer	Actuator + OpenTelemetry + agregadores (Zipkin, Jaeger, Prometheus, ELK)

Vamos começar pelas Métricas

- Para monitorar, gerenciar e diagnosticar uma aplicação em produção, precisamos responder perguntas como:
 - “Quanto CPU e memória RAM a aplicação está consumindo?”
 - “Quantas threads estão sendo usadas ao longo do tempo?”
 - “Qual a taxa de requisições que estão falhando?”
- **Métricas** são dados numéricos sobre a aplicação, medidos e agregados em intervalos regulares.
- Elas permitem:
 - Acompanhar a ocorrência de eventos (ex.: requisição HTTP recebida)
 - Contar itens (ex.: número de threads JVM alocadas)
 - Medir duração de tarefas (ex.: latência de consulta no banco)
 - Obter valores atuais de recursos (ex.: uso de CPU e RAM)
- Estas informações são valiosas para entender o **comportamento da aplicação**.

Métricas por meio do Spring Boot Actuator

- Spring Boot Actuator é um módulo do Spring Boot que adiciona:
 - Funcionalidades de monitoramento
 - Inspeção de runtime
 - Gerenciamento da aplicação
 - Ele expõe endpoints HTTP que fornecem informações valiosas sobre o estado da aplicação.

Métricas por meio do Spring Boot Actuator

Endpoint	Description
<code>/beans</code>	Shows a list of all the Spring beans managed by the application
<code>/configprops</code>	Shows a list of all the <code>@ConfigurationProperties</code> -annotated beans
<code>/env</code>	Shows a list of all the properties available to the Spring Environment
<code>/flyway</code>	Lists all the migrations run by Flyway and their statuses
<code>/health</code>	Shows information about the application's health
<code>/heapdump</code>	Returns a heap dump file
<code>/info</code>	Shows arbitrary application information
<code>/loggers</code>	Shows the configuration of all the loggers in the application and allows you to modify them
<code>/metrics</code>	Returns metrics about the application
<code>/mappings</code>	Lists all the paths defined in web controllers
<code>/prometheus</code>	Returns metrics about the application either in Prometheus or OpenMetrics format
<code>/sessions</code>	Lists all the active sessions managed by Spring Session and allows you to delete them
<code>/threaddump</code>	Returns a thread dump in JSON format

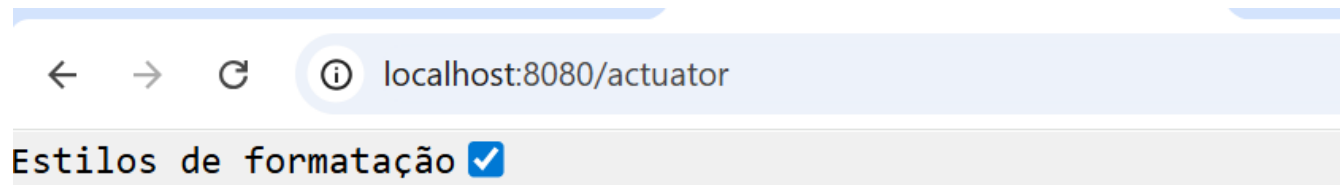
Métricas por meio do Spring Boot Actuator

- Spring Boot Actuator pode ser adicionado ao projeto via a seguinte dependência:

```
<dependency>  
  <groupId>org.springframework.boot</groupId>  
  <artifactId>spring-boot-starter-actuator</artifactId>  
  <version>3.4.5</version>  
</dependency>
```

Métricas por meio do Spring Boot Actuator

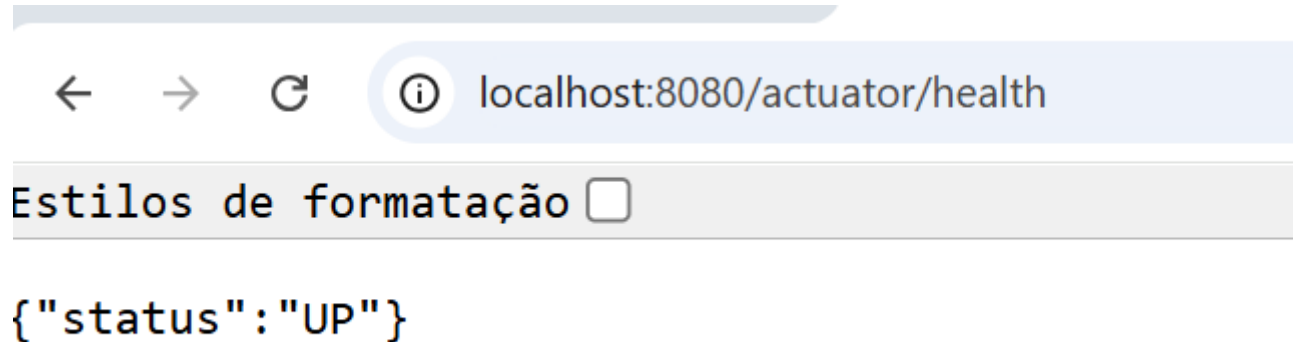
- Por padrão são liberadas as seguintes rotas:



```
{
  "_links": {
    "self": {
      "href": "http://localhost:8080/actuator",
      "templated": false
    },
    "health": {
      "href": "http://localhost:8080/actuator/health",
      "templated": false
    },
    "health-path": {
      "href": "http://localhost:8080/actuator/health/{*path}",
      "templated": true
    }
  }
}
```

Métricas por meio do Spring Boot Actuator

- Por padrão são liberadas as seguintes rotas:

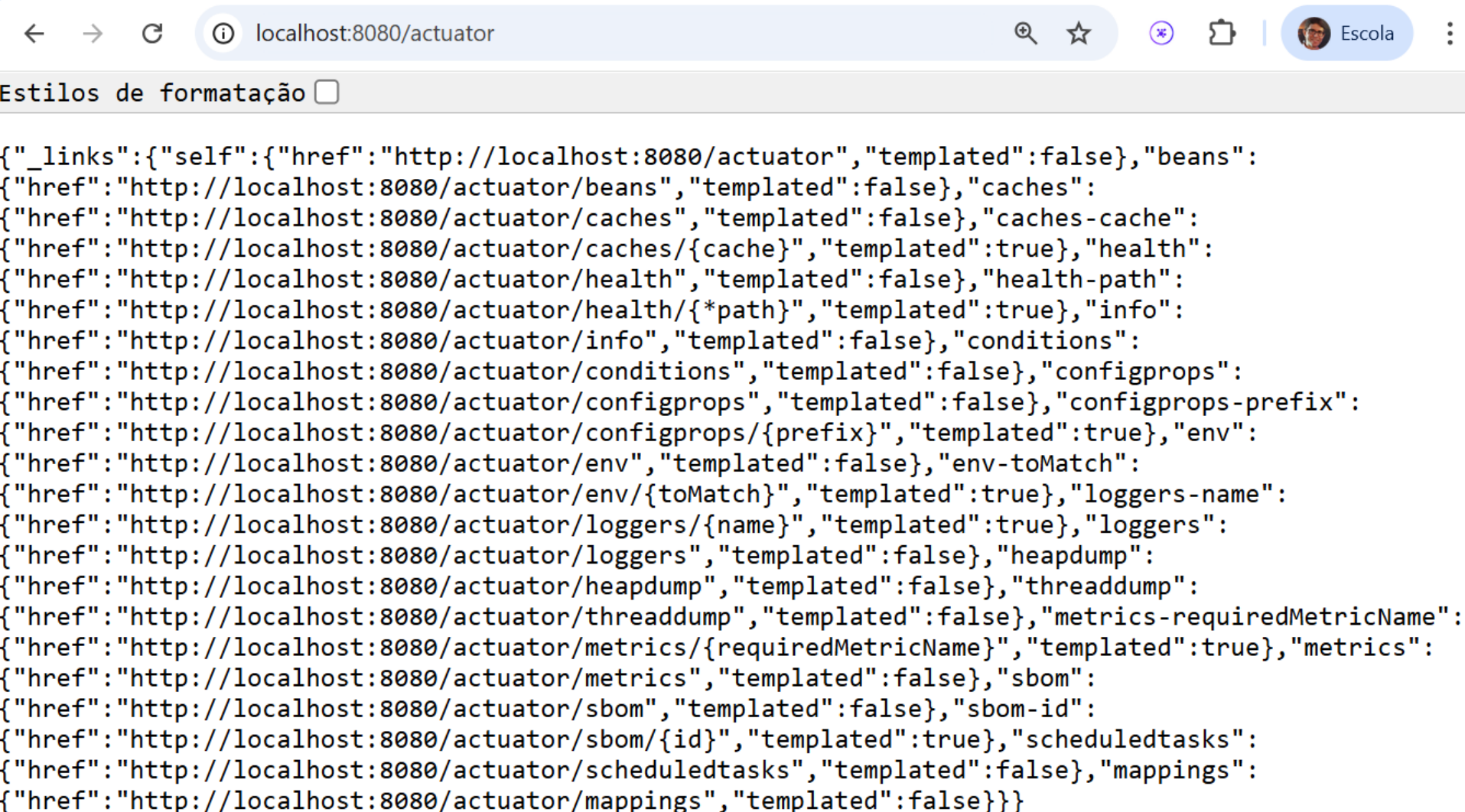


Métricas por meio do Spring Boot Actuator

- Mas, por meio da configuração, pode-se abrir outras rotas. No entanto, isso deve ser feito com cuidado quando a aplicação estiver em produção. Além de questões de segurança, também existem um incremento no volume de dados enviados pela sua aplicação, podendo haver impacto na questão de desempenho.

```
management.endpoints.web.exposure.include=*  
management.endpoint.health.show-details=always  
management.endpoint.health.show-components=always
```

Métricas por meio do Spring Boot Actuator



```
{
  "_links": {
    "self": {
      "href": "http://localhost:8080/actuator",
      "templated": false
    },
    "beans": {
      "href": "http://localhost:8080/actuator/beans",
      "templated": false
    },
    "caches": {
      "href": "http://localhost:8080/actuator/caches",
      "templated": false
    },
    "caches-cache": {
      "href": "http://localhost:8080/actuator/caches/{cache}",
      "templated": true
    },
    "health": {
      "href": "http://localhost:8080/actuator/health",
      "templated": false
    },
    "health-path": {
      "href": "http://localhost:8080/actuator/health/{*path}",
      "templated": true
    },
    "info": {
      "href": "http://localhost:8080/actuator/info",
      "templated": false
    },
    "conditions": {
      "href": "http://localhost:8080/actuator/conditions",
      "templated": false
    },
    "configprops": {
      "href": "http://localhost:8080/actuator/configprops",
      "templated": false
    },
    "configprops-prefix": {
      "href": "http://localhost:8080/actuator/configprops/{prefix}",
      "templated": true
    },
    "env": {
      "href": "http://localhost:8080/actuator/env",
      "templated": false
    },
    "env-toMatch": {
      "href": "http://localhost:8080/actuator/env/{toMatch}",
      "templated": true
    },
    "loggers-name": {
      "href": "http://localhost:8080/actuator/loggers/{name}",
      "templated": true
    },
    "loggers": {
      "href": "http://localhost:8080/actuator/loggers",
      "templated": false
    },
    "heapdump": {
      "href": "http://localhost:8080/actuator/heapdump",
      "templated": false
    },
    "threaddump": {
      "href": "http://localhost:8080/actuator/threaddump",
      "templated": false
    },
    "metrics-requiredMetricName": {
      "href": "http://localhost:8080/actuator/metrics/{requiredMetricName}",
      "templated": true
    },
    "metrics": {
      "href": "http://localhost:8080/actuator/metrics",
      "templated": false
    },
    "sbom": {
      "href": "http://localhost:8080/actuator/sbom",
      "templated": false
    },
    "sbom-id": {
      "href": "http://localhost:8080/actuator/sbom/{id}",
      "templated": true
    },
    "scheduledtasks": {
      "href": "http://localhost:8080/actuator/scheduledtasks",
      "templated": false
    },
    "mappings": {
      "href": "http://localhost:8080/actuator/mappings",
      "templated": false
    }
  }
}
```

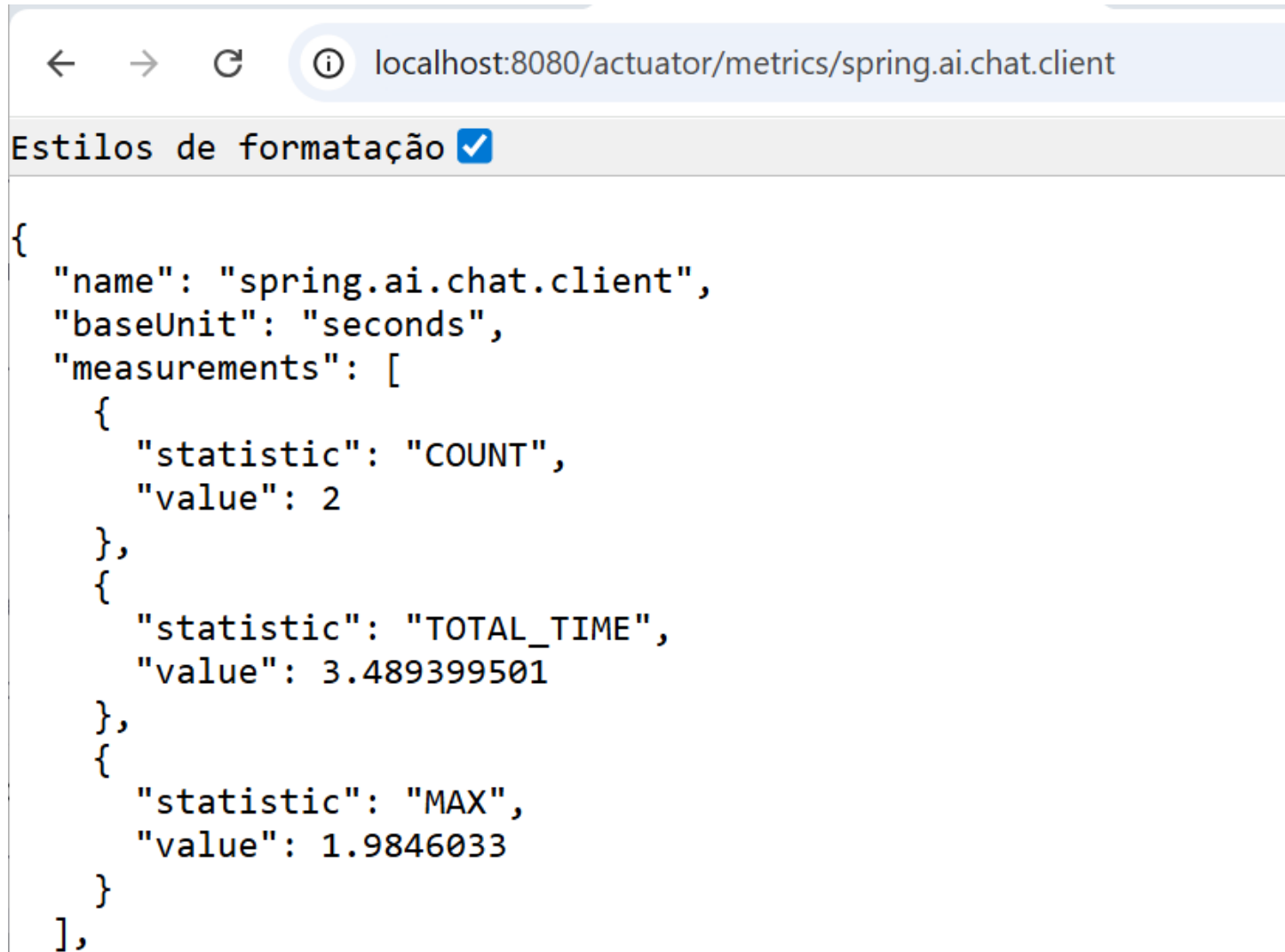
Métricas por meio do Spring Boot Actuator

← → ↻ ⓘ localhost:8080/actuator/metrics

Estilos de formatação ☒

```
{
  "names": [
    "application.ready.time",
    "application.started.time",
    "disk.free",
    "disk.total",
    "executor.active",
    "executor.completed",
    "executor.pool.core",
    "executor.pool.max",
    "spring.ai.advisor",
    "spring.ai.advisor.active",
    "spring.ai.chat.client",
    "spring.ai.chat.client.active",
    "system.cpu.count",
    "system.cpu.usage",
    "tomcat.sessions.active.current",
    "tomcat.sessions.active.max",
    "tomcat.sessions.alive.max",
    "tomcat.sessions.created",
    "tomcat.sessions.expired",
    "tomcat.sessions.rejected"
  ]
}
```

Métricas por meio do Spring Boot Actuator



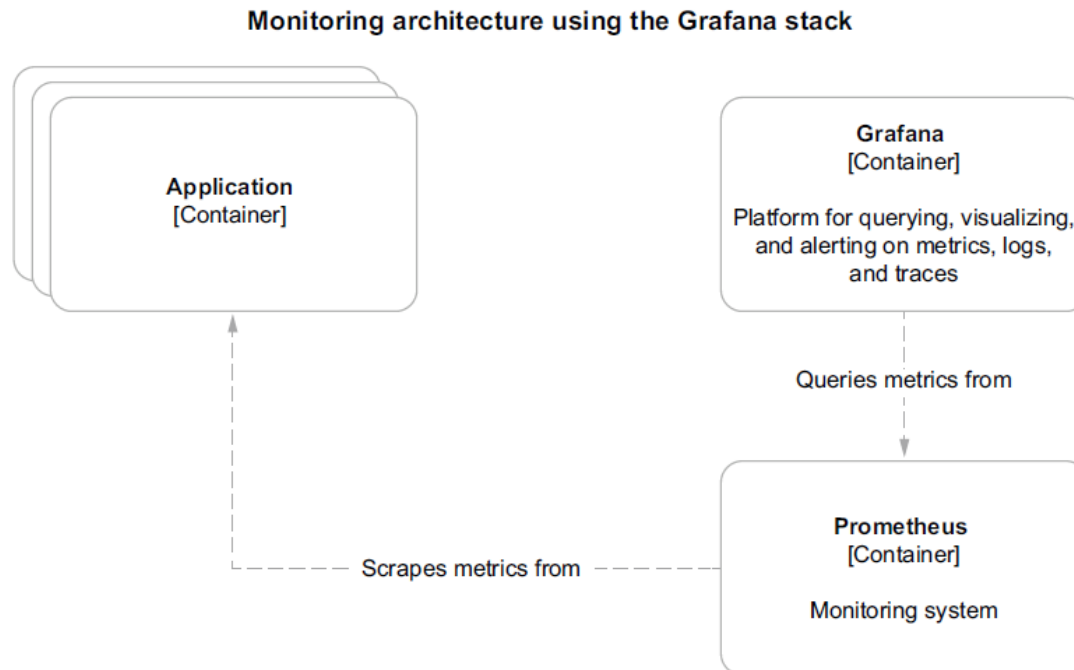
A screenshot of a web browser window displaying the metrics for the 'spring.ai.chat.client' endpoint. The address bar shows 'localhost:8080/actuator/metrics/spring.ai.chat.client'. Below the address bar, there is a toggle for 'Estilos de formatação' (Formatting styles) which is checked. The main content area displays a JSON object representing the metrics.

```
{
  "name": "spring.ai.chat.client",
  "baseUnit": "seconds",
  "measurements": [
    {
      "statistic": "COUNT",
      "value": 2
    },
    {
      "statistic": "TOTAL_TIME",
      "value": 3.489399501
    },
    {
      "statistic": "MAX",
      "value": 1.9846033
    }
  ]
},
```

<https://docs.spring.io/spring-ai/reference/observability/index.html>

Métricas por meio do Spring Boot Actuator

- Spring Boot Actuator é um módulo que fornece métricas interessantes, mas ele sozinho não permite monitorar uma aplicação de forma profissional. Para tanto, vamos utilizar a arquitetura abaixo:



Prometheus: O Guardião das Métricas

- **Prometheus** é um conjunto de ferramentas de monitoramento e alerta, totalmente open-source, criado originalmente na SoundCloud. Ele se tornou um padrão de mercado para monitoramento em ambientes de nuvem e microserviços, sendo o segundo projeto a se graduar na **Cloud Native Computing Foundation (CNCF)**, logo após o Kubernetes.

Prometheus: O Guardião das Métricas

- **Finalidade Principal:** Coletar e armazenar métricas como **séries temporais** (time-series data), ou seja, informações com carimbos de tempo.
- **Foco:** Confiabilidade e simplicidade. É projetado para ser o sistema que você usa para descobrir por que seus outros sistemas estão falhando.
- **Modelo de Coleta:** Predominantemente *pull-based* (baseado em "puxar" dados), onde o Prometheus ativamente busca (faz *scrape*) as métricas de endpoints HTTP expostos pelas aplicações.

Como Integrar o Spring com Prometheus ?

- Cada sistema de monitoramento (Prometheus, Datadog, Atlas da Netflix, etc.) tem sua própria biblioteca de instrumentação e seu próprio modelo de dados.
- Isso gera vários problemas:
 - **Vendor Lock-in (Aprisionamento Tecnológico):** Se uma aplicação fosse instrumentada diretamente com a biblioteca do Datadog, migrar para o Prometheus exigiria reescrever todo o código de métricas.
 - **Manutenção de Bibliotecas:** Frameworks como o **Spring Boot** precisavam manter módulos de integração separados e complexos para cada sistema de monitoramento que quisessem suportar.
 - **Inconsistência:** Não havia um padrão para os nomes e tipos de métricas, dificultando a criação de dashboards e alertas consistentes entre diferentes serviços.

Micrometer

- **Micrometer** é uma biblioteca de instrumentação de aplicações para a JVM. Sua principal função é atuar como uma **fachada de métricas**, que permite aos desenvolvedores coletar dados de performance de suas aplicações de uma maneira agnóstica ao sistema de monitoramento.
- **O objetivo principal:** "Instrumente uma vez, envie para qualquer lugar."

Micrometer

- O Spring Boot Actuator utiliza o Micrometer para a instrumentação e fornece auto-configuração para gerar métricas para uma vasta gama de tecnologias.
- O Micrometer atua como uma fachada, permitindo que você integre com diferentes sistemas de monitoramento (como Prometheus, Datadog, etc.) com o mínimo de esforço.

```
<dependency>  
  <groupId>io.micrometer</groupId>  
  <artifactId>micrometer-registry-prometheus</artifactId>  
  <scope>runtime</scope>  
</dependency>
```

Coleta de Métricas via Actuator

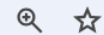
- Como dito, o Prometheus utiliza uma estratégia baseada em pull (puxar) como padrão.
- Isso significa que a instância do Prometheus "raspa" (coleta) métricas da sua aplicação em intervalos de tempo regulares.
- O Prometheus acessa um endpoint HTTP específico exposto pela sua aplicação Spring Boot para buscar os dados.
- Endpoint no Spring Boot: `/actuator/prometheus`.

Coleta de Métricas via Actuator

- Ao acessar o endpoint `/actuator/prometheus`, você não verá um JSON como em outros endpoints do Actuator.
- Em vez disso, as métricas são apresentadas em um formato de texto simples, específico para o Prometheus.
 - `# HELP`: Fornece uma breve descrição da métrica.
 - `# TYPE`: Define o tipo da métrica (ex: gauge, counter).
 - Um **Gauge** representa um "retrato instantâneo" (snapshot) de uma medida em um determinado momento(velocímetro de um carro).
 - Um **Counter** é ideal para medir a ocorrência de eventos cumulativos(odômetro de um carro).

Coleta de Métricas via Actuator

localhost:8090/actuator/prometheus



```
# HELP application_ready_time_seconds Time taken for the application to be ready to service requests
# TYPE application_ready_time_seconds gauge
application_ready_time_seconds{main_application_class="br.imd.ufrn.betaservice.BetaserviceApplication"} 3.859
# HELP application_started_time_seconds Time taken to start the application
# TYPE application_started_time_seconds gauge
application_started_time_seconds{main_application_class="br.imd.ufrn.betaservice.BetaserviceApplication"} 3.853
# HELP disk_free_bytes Usable space for path
# TYPE disk_free_bytes gauge
disk_free_bytes{path="C:\\Users\\nelio\\OneDrive\\Disciplinas\\Aulas Gerais de Programação
Distribuída\\Observabilidade\\Aplicacoes\\betaservice\\"} 2.99705888768E11
# HELP disk_total_bytes Total space for path
# TYPE disk_total_bytes gauge
disk_total_bytes{path="C:\\Users\\nelio\\OneDrive\\Disciplinas\\Aulas Gerais de Programação
Distribuída\\Observabilidade\\Aplicacoes\\betaservice\\"} 4.91707691008E11
# HELP executor_active_threads The approximate number of threads that are actively executing tasks
# TYPE executor_active_threads gauge
executor_active_threads{name="applicationTaskExecutor"} 0.0
# HELP executor_completed_tasks_total The approximate total number of tasks that have completed execution
# TYPE executor_completed_tasks_total counter
executor_completed_tasks_total{name="applicationTaskExecutor"} 0.0
# HELP executor_pool_core_threads The core number of threads for the pool
# TYPE executor_pool_core_threads gauge
executor_pool_core_threads{name="applicationTaskExecutor"} 8.0
# HELP executor_pool_max_threads The maximum allowed number of threads in the pool
# TYPE executor_pool_max_threads gauge
executor_pool_max_threads{name="applicationTaskExecutor"} 2.147483647E9
# HELP executor_pool_size_threads The current number of threads in the pool
# TYPE executor_pool_size_threads gauge
```

Instalando o Prometheus



Prometheus

[Docs](#)

[Download](#)

[Community](#)

Operating System

popular

Architecture

popular

prometheus

The Prometheus monitoring system and time series database.

3.5.0-rc.0 / 2025-06-25 **PRE-RELEASE**

File name	OS	Arch	Size
prometheus-3.5.0-rc.0.darwin-amd64.tar.gz	darwin	amd64	117.26 MiB
prometheus-3.5.0-rc.0.darwin-arm64.tar.gz	darwin	arm64	113.01 MiB
prometheus-3.5.0-rc.0.linux-amd64.tar.gz	linux	amd64	115.48 MiB
prometheus-3.5.0-rc.0.windows-amd64.zip	windows	amd64	119.22 MiB

3.4.2 / 2025-06-26 **LATEST**

File name	OS	Arch	Size
prometheus-3.4.2.darwin-amd64.tar.gz	darwin	amd64	113.51 MiB
prometheus-3.4.2.darwin-arm64.tar.gz	darwin	arm64	109.42 MiB
prometheus-3.4.2.linux-amd64.tar.gz	linux	amd64	111.85 MiB
prometheus-3.4.2.windows-amd64.zip	windows	amd64	115.53 MiB

Instalando o Prometheus

- No Linux:
 - Baixar o arquivo do site <https://prometheus.io/download/> :
 - `wget https://github.com/prometheus/prometheus/releases/download/v3.5.0-rc.0/prometheus-3.5.0-rc.0.linux-amd64.tar.gz`
 - *Descompactar o arquivo:*
 - `tar -xvzf prometheus-3.5.0-rc.0.linux-amd64.tar.gz`
 - *Editar o arquivo **prometheus.yml***
 - *Executar o servidor*
 - `./prometheus`
- No Windows
 - Baixar e descompactar do site:
<https://prometheus.io/download/>
 - *Executar o servidor `prometheus.exe`*

Instalando o Prometheus

! prometheus.yml X

C: > Users > nelio > OneDrive > Disciplinas > Aulas Gerais de Programação Distribuída > Observabilidade > prometheus-3.4.2.windows-amd

```
1  global:
2    scrape_interval: 2s # Set the scrape interval to every 15 seconds. Default is every 1 minute.
3    evaluation_interval: 2s # Evaluate rules every 15 seconds. The default is every 1 minute.
4
5  # Alertmanager configuration
6  alerting:
7    alertmanagers:
8      - static_configs:
9        - targets:
10          # - alertmanager:9093
11
12  # Load rules once and periodically evaluate them according to the global 'evaluation_interval'.
13  rule_files:
14    # - "first_rules.yml"
15    # - "second_rules.yml"
16
17  scrape_configs:
18    - job_name: 'EurekaServer'
19      metrics_path: '/actuator/prometheus'
20      static_configs:
21        - targets: ['localhost:8761']
22    - job_name: 'APIGateway'
23      metrics_path: '/actuator/prometheus'
24      static_configs:
25        - targets: ['localhost:8080']
26
27    - job_name: 'AlphaService'
28      metrics_path: '/actuator/prometheus'
29      static_configs:
30        - targets: ['localhost:8070']
31
32    - job_name: 'BetaService'
33      metrics_path: '/actuator/prometheus'
34      static_configs:
35        - targets: ['localhost:8090']
36
37    - job_name: 'prometheus'
38      static_configs:
39        - targets: ["localhost:9090"]
40      labels:
41        app: "prometheus"
42
```

Grafana

- Grafana é uma plataforma de visualização e análise de dados de código aberto. Ele permite que você consulte, visualize, alerte e entenda suas métricas, não importa onde elas estejam armazenadas.
 - Finalidade Principal: Transformar dados brutos de séries temporais (como os do Prometheus) em dashboards bonitos, informativos e fáceis de entender.
 - Não armazena dados: O Grafana não é um banco de dados. Ele se conecta a diversas fontes de dados (Data Sources) para buscar as informações que exibe.
 - Padrão de Mercado: É a ferramenta de visualização mais popular no mundo da observabilidade, especialmente em ambientes de nuvem e microserviços.

Instalando o Grafana

 **Grafana Labs**

ProductsOpen SourceSolutionsLearnDocsPricing

Grafana

OverviewGrafana projectGrafana AlertingDownload

Download Grafana

 **Grafana Cloud**

You can use Grafana Cloud to avoid installing, maintaining, and scaling your own instance of Grafana. [Create](#) access to 10k series Prometheus metrics, 50GB logs, 50GB traces, & more.

Version:

12.0.2

Edition:

Enterprise

License:

[Grafana Labs License](#)

Release Date:

June 17, 2025

Release Info:

[What's New In Grafana 12.0.2](#)


Linux

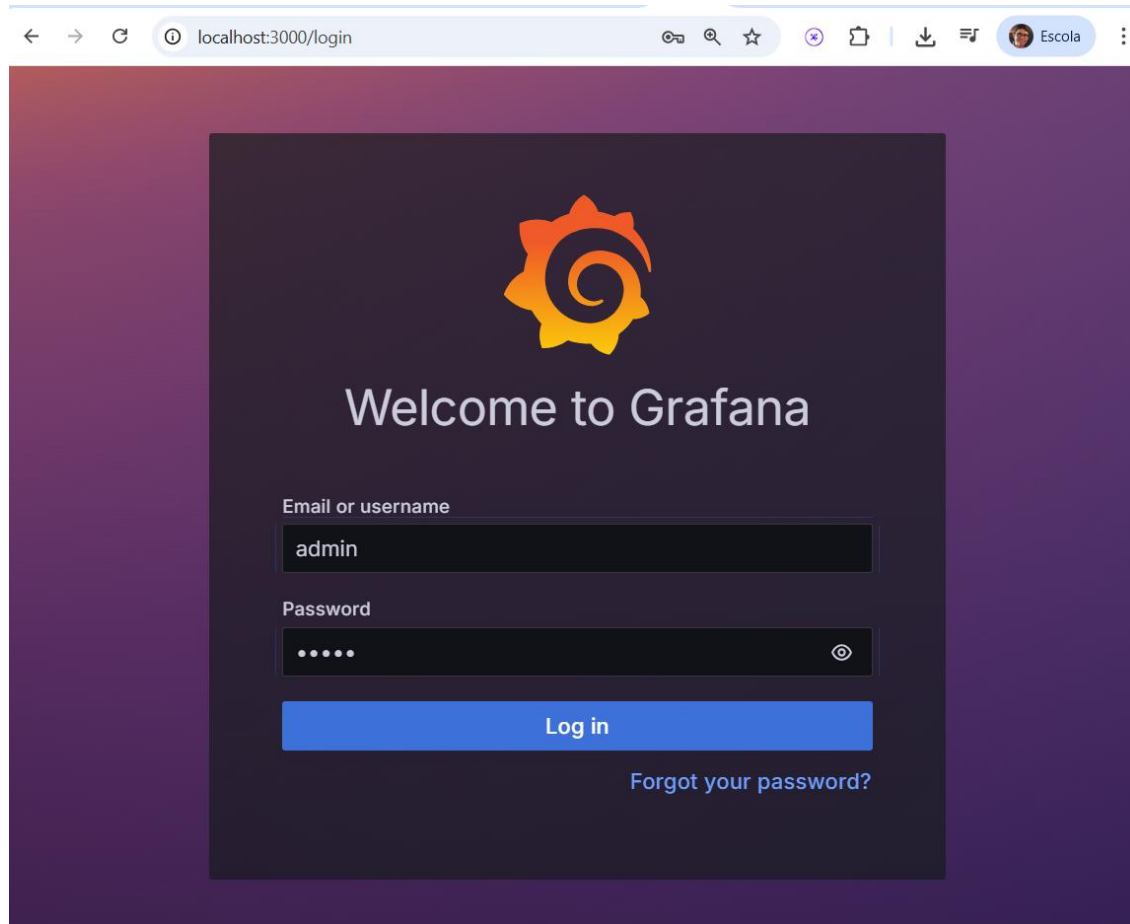

Windows


Mac


Docker


Linux on ARM64

Grafana

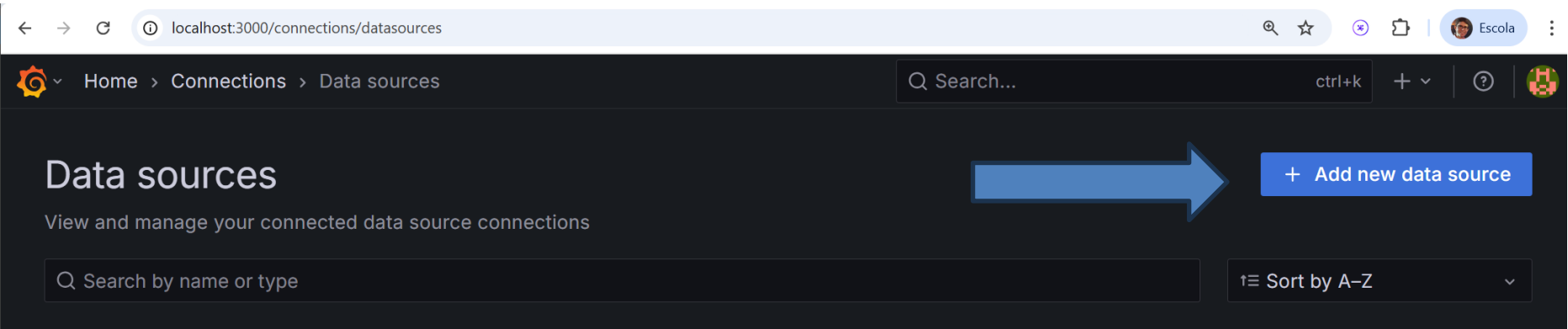


Acessar <http://localhost:3000/login> com usuário: admin e senha: admin

Configurar fonte de Dados do Grafana

The image shows a screenshot of the Grafana web interface in a browser. The address bar at the top displays the URL: `localhost:3000/?orgId=1&from=now-6h&to=now&timezone=browser`. The left sidebar menu is open, showing the following items: Home, Bookmarks, Starred, Dashboards, Explore, Drilldown (with a 'New!' badge), Alerting, Connections, Add new connection, Data sources (highlighted with a blue arrow), and Administration. The main content area on the right has a search bar and links for 'Need help?' (Documentation, Tutorials, Community, Publi). Below these, there are three cards: 'Add your first data source' (labeled 'COMPLETE'), 'Create your first dashboard' (labeled 'COMPLETE'), and a partially visible card about 'Grafana fundamentals'.

Configurar fonte de Dados do Grafana



Configurar fonte de Dados do Grafana

localhost:3000/connections/datasources/new

Home > Connections > Data sources > Add data source

Search...

Add data source

Choose a data source type

Filter by name or type

Cancel

Time series databases



Prometheus
Open source time series database & alerting
Core

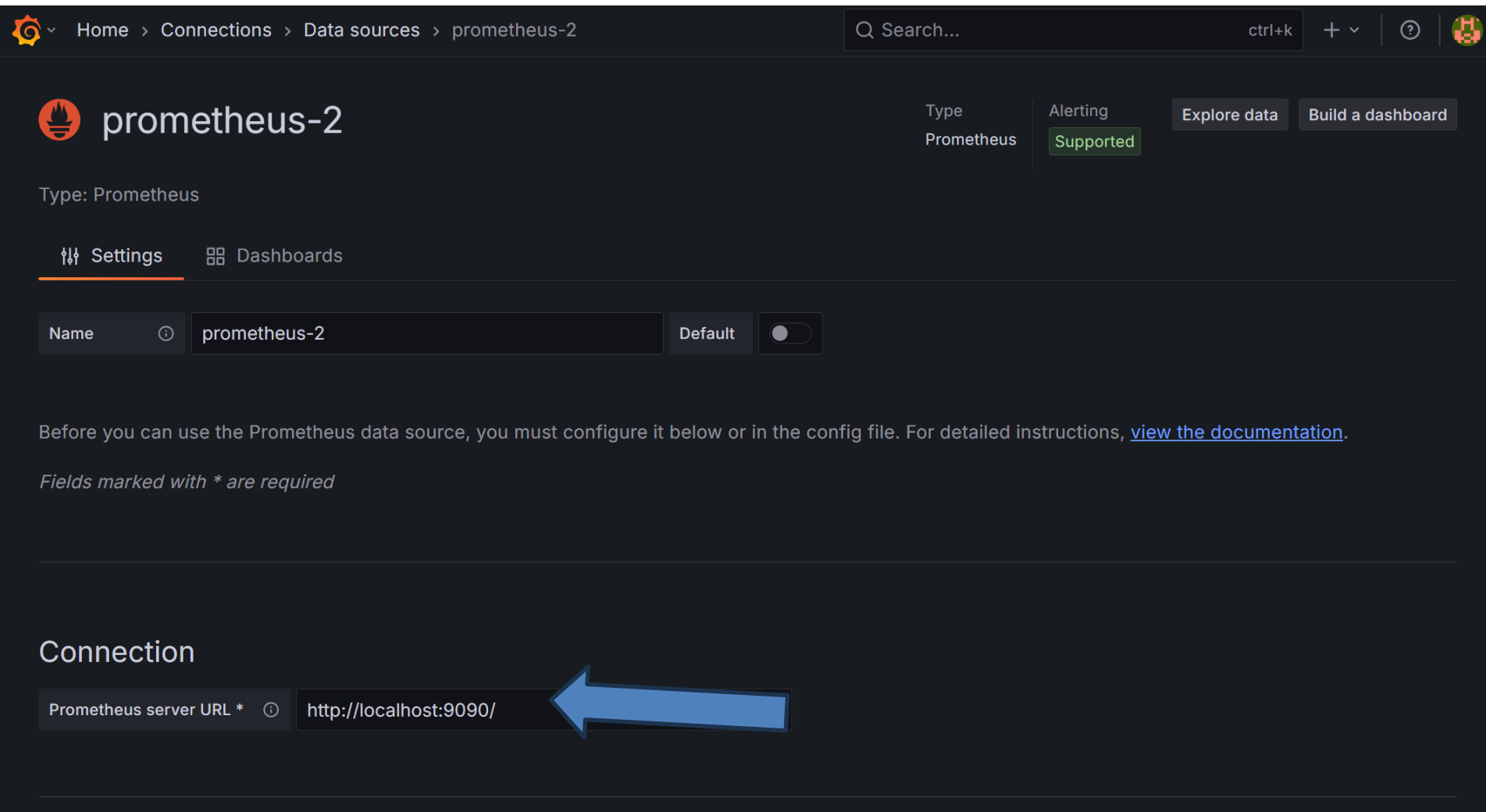


Graphite
Open source time series database
Core



InfluxDB
Open source time series database
Core

Configurar fonte de Dados do Grafana



The screenshot shows the Grafana web interface. At the top, a blue banner contains the title 'Configurar fonte de Dados do Grafana'. Below this, the navigation bar shows the path 'Home > Connections > Data sources > prometheus-2'. A search bar and utility icons are on the right. The main header for the 'prometheus-2' data source includes the Prometheus logo, the name 'prometheus-2', and tabs for 'Type' (Prometheus), 'Alerting' (Supported), 'Explore data', and 'Build a dashboard'. Below the header, there are tabs for 'Settings' (active) and 'Dashboards'. The 'Settings' tab shows a form with a 'Name' field containing 'prometheus-2', a 'Default' toggle switch, and a 'Prometheus server URL' field containing 'http://localhost:9090/'. A large blue arrow points to the URL field. Below the form, there is a note about configuration requirements and a link to the documentation.

Home > Connections > Data sources > prometheus-2

Search... ctrl+k + ? 

 prometheus-2

Type: Prometheus

Type: Prometheus Alerting Supported Explore data Build a dashboard

Settings Dashboards

Name ⓘ prometheus-2 Default ☐

Before you can use the Prometheus data source, you must configure it below or in the config file. For detailed instructions, [view the documentation](#).

Fields marked with * are required

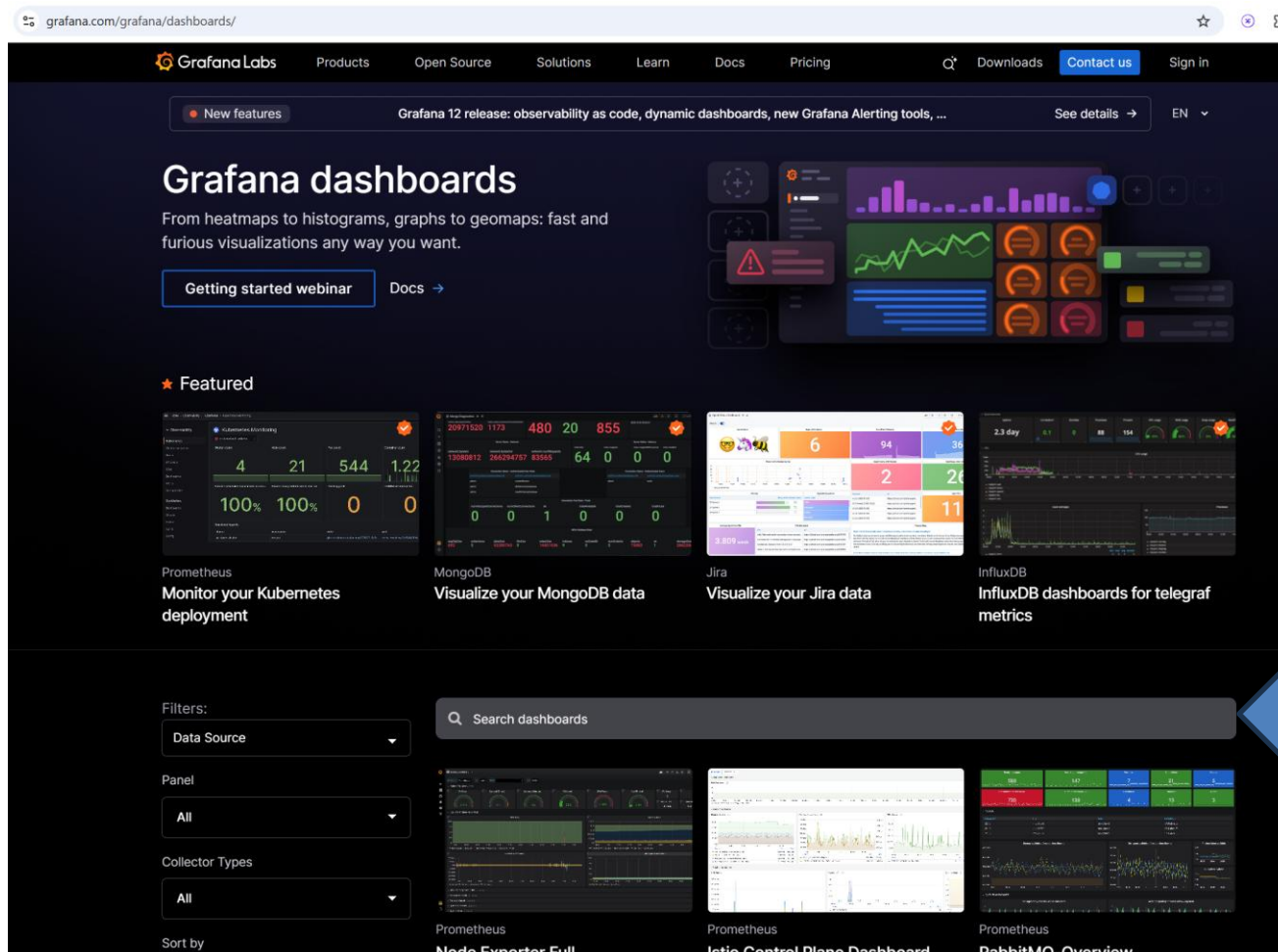
Connection

Prometheus server URL * ⓘ http://localhost:9090/

Forneça o endereço do Prometheus e no final do form, clicar em “Save & Test”

Importando Dashboard no Grafana

- Visite a página: <https://grafana.com/grafana/dashboards/> e procure por dashboards para Spring



Escolha um e copie o URL link

JVM (Micrometer)

Dashboard for Micrometer instrumented applications (Java, Spring Boot, Micronaut)

Application: micrometer2 - Instance: [redacted] Restart Detection: [on]

Quick Facts

- Uptime: 29.6 min
- Start time: 23:10:00
- Heap used: 54.75%
- Non-Heap used: 73.77%

Key Performance Indicators (KPIs)

- Rate: [Line chart showing requests per second]
- Errors: [Line chart showing error rate]
- Duration: [Line chart showing request duration]
- Utilisation: [Line chart showing system utilisation]

JVM Memory

- JVM Heap: [Line chart showing heap memory usage]
- JVM Non-Heap: [Line chart showing non-heap memory usage]
- JVM Total: [Line chart showing total memory usage]
- JVM Process Memory: [Line chart showing process memory usage]

JVM Misc

- CPU Usage: [Line chart showing CPU usage]
- Load: [Line chart showing system load]
- Threads: [Line chart showing thread count]
- Thread States: [Line chart showing thread states]

Java Virtual Machine (JVM)

Grafana Labs solution

Easily monitor a Java virtual machine, which allows computers to run Java programs, with Grafana Cloud's out-of-the-box monitoring solution.

[Learn more](#)

Get this dashboard

1

Sign up for Grafana Cloud

[Create free account](#)

2

Import the dashboard template

[Copy ID to clipboard](#)

or

Importando Dashboard

← → ↻ localhost:3000/dashboards 🔑 🔍 ☆ + 📁 ⬇️ 📑 Escola ⋮

⚙️ Home > Dashboards 🔍 + ▾ ⓘ 🧑

Dashboards

Create and manage dashboards to visualize your data

🔍 Search for dashboards and folders

🏷️ Filter by tag ▾ ☐ Starred 📁 ☰ ⌵ Sort

New ^

- New dashboard
- New folder
- Import



You haven't created any dashboards yet

+ Create dashboard

localhost:3000/dashboard/import

Importando Dashboard

 Home > Dashboards > Import dashboard

Import dashboard

Import dashboard from file or Grafana.com


Upload dashboard JSON file
Drag and drop here or click to browse
Accepted file types: .json, .txt

Find and import dashboards for common applications at grafana.com/dashboards

Load

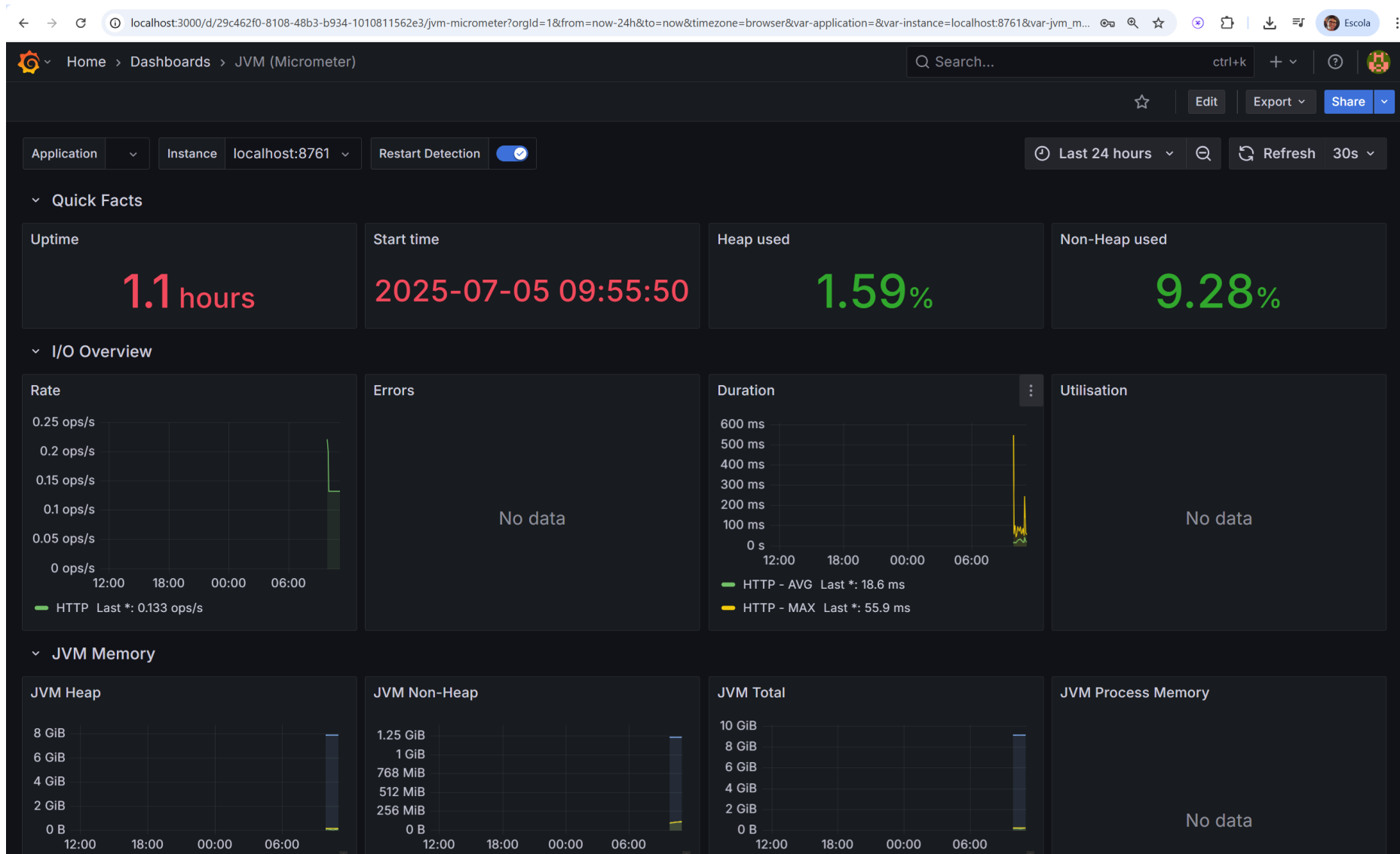
Import via dashboard JSON model

```
{
  "title": "Example - Repeating Dictionary variables",
  "uid": "_OHnEoN4z",
  "panels": [...]
  ...
}
```

Load

Cancel

Grafana Dashboard para Spring Apps



Por que Logs e Métricas Não São Suficientes?

- Logs e métricas são valiosos, mas têm uma limitação fundamental em arquiteturas de nuvem: **não correlacionam dados através das fronteiras das aplicações.**
- **O Desafio:** Uma única requisição de usuário geralmente passa por múltiplos serviços. Como podemos rastrear essa jornada de ponta a ponta?
- **Solução Inicial:** Gerar um **ID de correlação** na borda do sistema e passá-lo entre os serviços para rastrear logs.
- **A Evolução:** Essa ideia, levada adiante, nos leva ao **Rastreamento Distribuído**, uma técnica para rastrear requisições, localizar erros e solucionar problemas de performance em sistemas distribuídos.

Os 3 Pilares: Trace, Span e Tags

- O Rastreamento Distribuído se baseia em três conceitos principais:
 - **Trace (Rastreamento):** Representa a jornada completa de uma requisição ou transação através do sistema. É identificado por um trace ID único e é composto por um ou mais spans.
 - **Span (Intervalo):** Representa um único passo ou unidade de trabalho dentro da requisição (ex: uma chamada HTTP, uma consulta ao banco). É caracterizado por um tempo de início e fim e identificado unicamente pela combinação do trace ID e um span ID.
 - **Tags (Etiquetas):** São metadados que fornecem contexto adicional sobre um span, como a URI da requisição, o nome do usuário logado ou um identificador do tenant.

Visualizando o Fluxo

- Vamos considerar uma requisição para buscar livros, que passa pelo gateway e chega ao serviço de catálogo.
- O Trace completo dessa operação envolveria pelo menos três Spans:
 - Span 1 (Api Gateway Service): Recebe a requisição HTTP inicial do cliente.
 - Span 2 (Api Gateway Service): Roteia a requisição para o Alpha Service.
 - Span 3 (Alpha Service): Processa a requisição roteada e envia uma requisição para Beta Service.
 - Span 4 (Beta Service): Processa a requisição roteada para buscar os dados.

Padrões e Backends

- A implementação do Rastreamento Distribuído envolve a escolha de dois componentes:
 - um **Padrão** para gerar e propagar os traces,
 - e um **Backend** para coletá-los e armazená-los.

Padrões e Protocolos (Geração e Propagação)

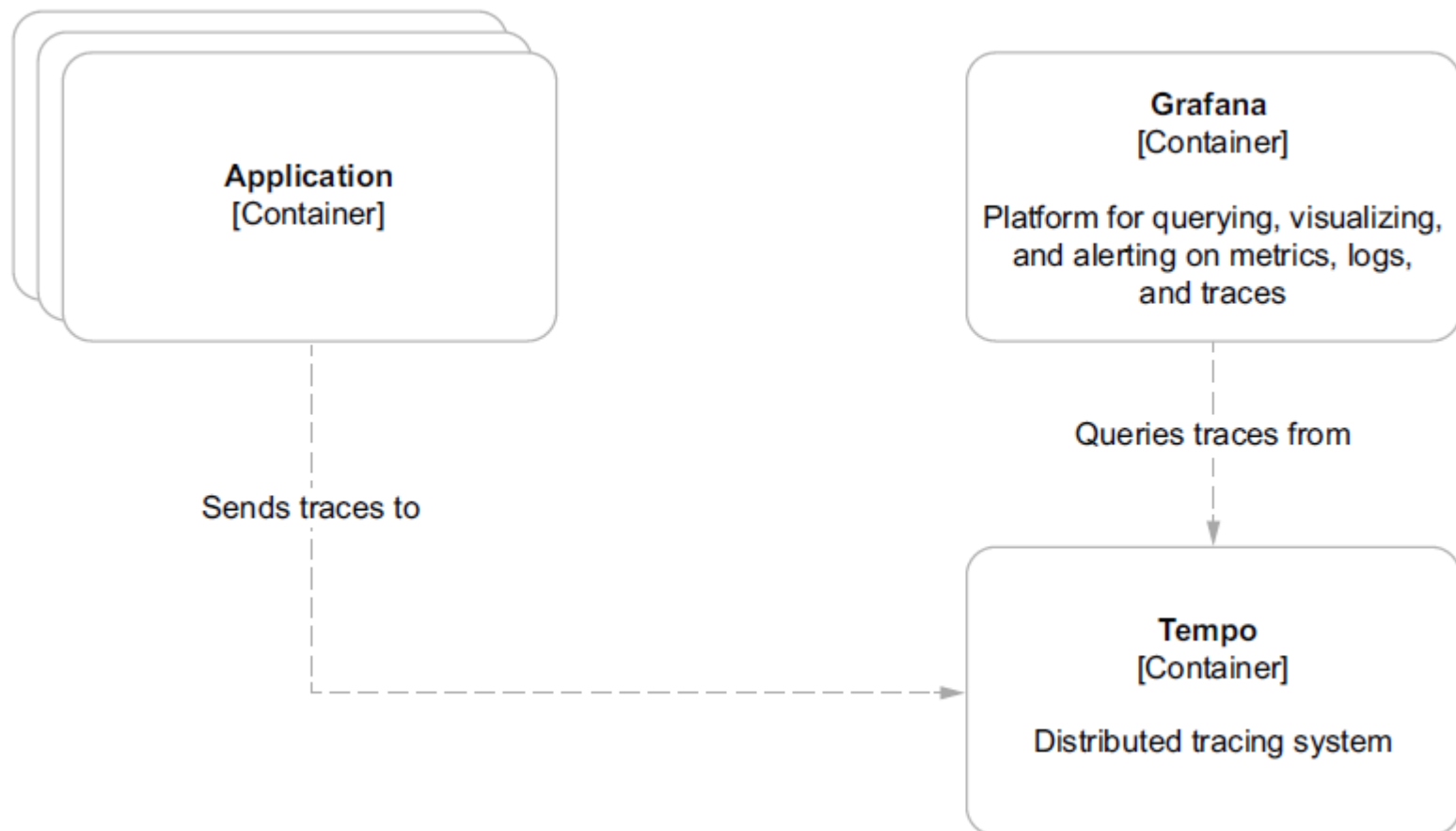
- Estes são os protocolos e formatos que definem como os spans e traces são enviados a backends de observabilidade:
 - **OTLP (OpenTelemetry Protocol):** Protocolo oficial do OpenTelemetry. Usa gRPC ou HTTP/Protobuf/JSON.
 - **Jaeger gRPC / Thrift:** Usado pelo Jaeger Collector. Exportação em gRPC ou Thrift compactado.
 - **Zipkin HTTP:** Formato JSON via POST HTTP.
 - **OpenCensus Agent:** Antecessor ao OpenTelemetry. Também exportava via gRPC.
 - **Google Cloud Trace, AWS X-Ray, Azure Monitor:** Usam protocolos proprietários. OpenTelemetry fornece exporters oficiais.

Backends de Rastreamento (Coleta, Armazenamento e Visualização)

- Estes são os sistemas que recebem, armazenam e permitem consultar os dados de trace gerados pelas aplicações.
 - **Grafana Tempo:** Projetado para alta escala com baixo custo operacional. Utiliza uma estratégia baseada em push e se integra perfeitamente ao ecossistema Grafana para visualização.
 - **Zipkin:** A Solução Completa e Popular: Além de um formato, Zipkin também é um backend completo com sua própria interface de usuário para visualização. É uma opção robusta e muito utilizada na comunidade.
 - **Outros Backends:** Existem diversas outras ferramentas comerciais e open-source, como Jaeger (também um projeto da CNCF) e Datadog APM.

Interoperabilidade: É importante notar que muitos backends, como o Tempo, são projetados para serem flexíveis e suportam múltiplos formatos de ingestão, incluindo OpenTelemetry e Zipkin.

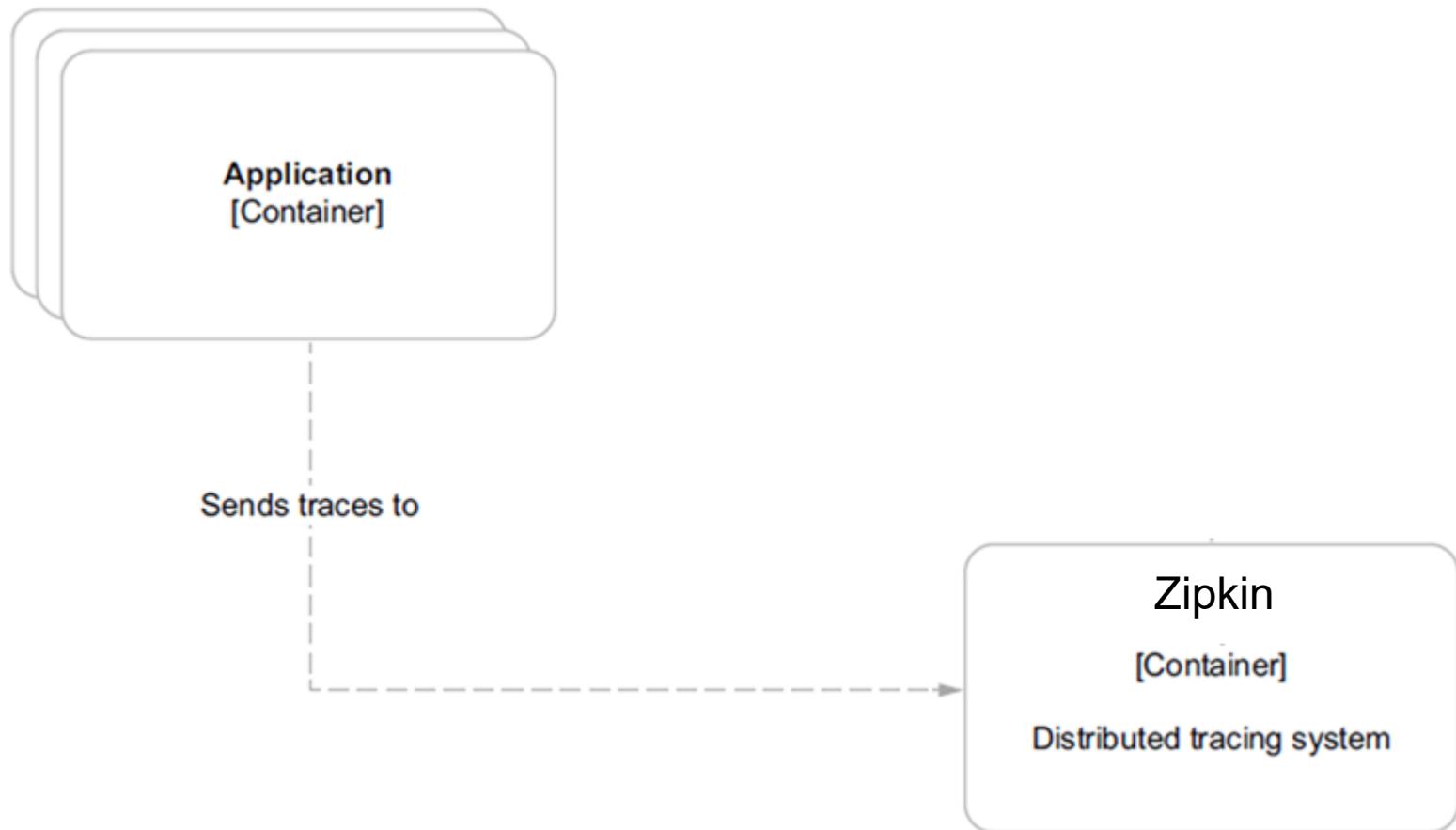
Arquitetura de Rastreamento Distribuído



De Spring Cloud Sleuth para Micrometer Tracing

- A forma de implementar o rastreamento em aplicações Spring evoluiu.
 - O Tradicional: Spring Cloud Sleuth
 - Projeto que fornece auto-configuração para rastreamento distribuído em aplicações Spring Boot, abstraindo bibliotecas específicas como OpenZipkin.
 - O atual: Micrometer Tracing.
 - O projeto Sleuth não será mais desenvolvido a partir do Spring Framework 6 e Spring Boot 3. Seu núcleo foi doado para o Micrometer, criando o novo subprojeto Micrometer Tracing. O objetivo é fornecer uma fachada neutra de fornecedor para traces, assim como o Micrometer já faz para métricas. Isso se tornará um aspecto central de todas as bibliotecas Spring como parte da iniciativa Spring Observability.
 - O futuro: OpenTelemetry Protocol

Nossa arquitetura



Zipkin

- O Zipkin foi originalmente desenvolvido no Twitter, com base no conceito de um artigo do Google que descrevia o Dapper, o depurador de aplicações distribuídas desenvolvido internamente pelo Google.
- Ele gerencia tanto a coleta quanto a consulta desses dados.
- Para usar o Zipkin, as aplicações são instrumentadas para reportar dados de tempo (timing data) a ele.

Zipkin

- Se você está solucionando problemas de latência ou erros em um ecossistema, pode filtrar ou ordenar todos os traces com base na aplicação, duração do trace, anotação ou carimbo de tempo (timestamp).
- Analisando esses traces, você pode decidir quais componentes não estão performando conforme o esperado e pode corrigi-los.

Zipkin

- O Zipkin possui 4 módulos:
 - **Collector (Coletor)** – Assim que um componente envia os dados de trace, eles chegam ao daemon do coletor Zipkin. Aqui, os dados de trace são validados, armazenados e indexados para consulta pelo coletor Zipkin.
 - **Storage (Armazenamento)** – Este módulo armazena e indexa os dados de consulta no backend. Cassandra, ElasticSearch e MySQL são suportados.
 - **Search (Busca)** – Este módulo fornece uma API JSON simples para encontrar e recuperar os traces armazenados no backend. O principal consumidor desta API é a Web UI.
 - **Web UI (Interface de Usuário)** – Uma interface de usuário muito boa para visualizar os traces.

Spring Cloud Sleuth with Zipkin

- Zipkin é utilizado para rastrear as chamadas entre os serviços;
- Para usar, faça o seguinte em todos os serviços:

```
<dependency>
  <groupId>io.micrometer</groupId>
  <artifactId>micrometer-tracing-bridge-otel</artifactId>
</dependency>
```

```
<!--Reporter/Zipkin
with OpenTelemetry: Reports traces to Zipkin-->
```

```
<dependency>
  <groupId>io.opentelemetry</groupId>
  <artifactId>opentelemetry-exporter-zipkin</artifactId>
</dependency>
```

```
management:
  tracing:
    sampling:
      probability: 1.0
```

Instalando Zipkin

- Instalando Zipkin
 - `curl -sSL https://zipkin.io/quickstart.sh | bash -s`
 - `java -jar zipkin.jar`

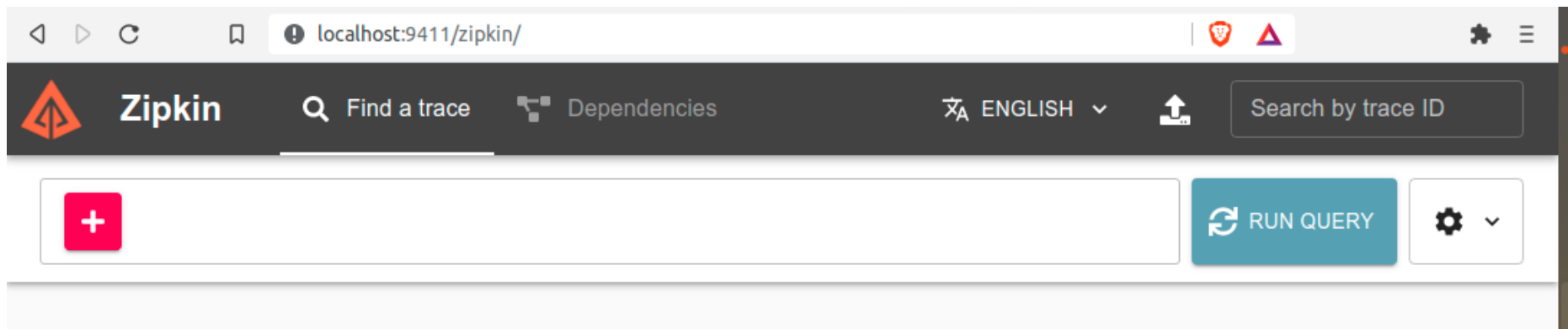
Zipkin

```
nelio@Nelio-PC:~$ java -jar zipkin.jar
```

```
2021-04-12 16:10:24.288 INFO [/] 10002 --- [oss-http-*:9411] c.l.a.s.Server : Serving HTTP at /0:0:0:0:0:0:0:0:9411 - http://127.0.0.1:9411/
```

```
2021-04-12 16:10:24.288 INFO [/] 10002 --- [oss-http-*:9411] c.l.a.s.Server : Serving HTTP at /0:0:0:0:0:0:0:0:9411 - http://127.0.0.1:9411/
```

Zipkin

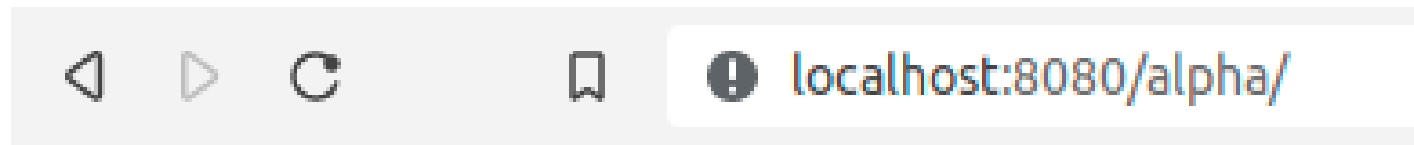


Executando o rastreamento

Instances currently registered with Eureka

Application	AMIs	Availability Zones	Status
SERVICEALPHA	n/a (1)	(1)	UP (1) - 10.0.0.167:servicealpha:8081
SERVICEBETA	n/a (1)	(1)	UP (1) - 10.0.0.167:servicebeta:8091
SERVICEGATEWAYAPI	n/a (1)	(1)	UP (1) - 10.0.0.167:ServiceGatewayAPI:8080

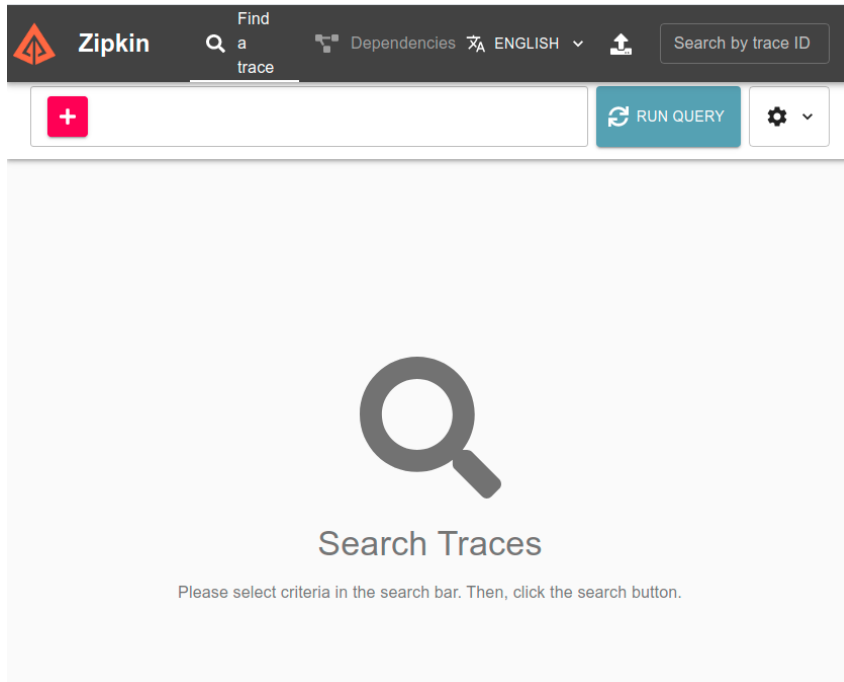
Executando o rastreamento



OK from 8091

Executando o rastreamento

Acessar o link para ver o trace:
<http://localhost:9411/zipkin/>



Executando o rastreamento



 RUN QUERY



1 Result

EXPAND ALL

COLLAPSE ALL

Service filters



Root

Start Time

Spans

↓ Duration



servicegatewayapi: get

a few seconds ago (02/02 11:26:04:428)

6

16.248ms

SHOW

Trace ID: 09a08af305b0f68d

servicegatewayapi (3)

servicealpha (2)

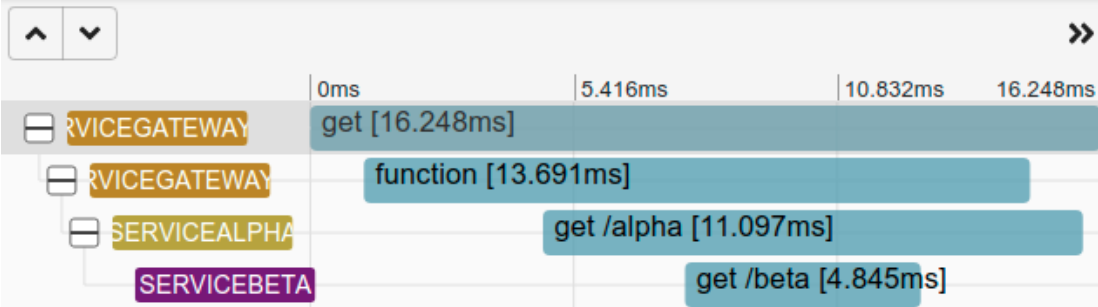
servicebeta (1)

Executando o rastreamento

SERVICEGATEWAYAPI: get

Duration: 16.248ms Services: 3 Depth: 4 Total Spans: 4 Trace ID: 09a08af305b0f68d

 DOWNLOAD JSON



SERVICEGATEWAYAPI

get

Span ID: 09a08af305b0f68d Parent ID: None

Annotations



SHOW ALL ANNOTATIONS

Tags

http.method
GET

http.path
/alpha/

Executando o rastreamento

The image displays the Zipkin web interface. At the top, the Zipkin logo is on the left, followed by navigation links: "Find a trace" (with a magnifying glass icon), "Dependencies" (with a network icon), and "ENGLISH" (with a flag icon and a dropdown arrow). On the far right of the top bar is a "Search by trace ID" input field. Below the top bar, there are two date-time pickers: "Start Time" set to "02/01/2022 11:28:18" and "End Time" set to "02/02/2022 11:28:18", separated by a minus sign. A search icon is to the right of the end time picker. Below these filters is a "Select..." dropdown menu. The main area of the interface shows a trace visualization with three nodes connected by lines. The nodes are labeled "servicegatewayapi", "servicealpha", and "servicebeta".

Zipkin

Find a trace

Dependencies

ENGLISH

Search by trace ID

Start Time
02/01/2022 11:28:18

End Time
02/02/2022 11:28:18

Select...

servicegatewayapi

servicealpha

servicebeta

Executando o rastreamento

